

Strings and Basic I/O

Text values and observable program behavior

Gregory S. DeLozier, PhD

Breaking Changes

Earlier code	Change needed
<code>print x</code>	<code>print(x)</code>
Variables <code>input</code> , <code>number</code> , <code>string</code>	Rename: now keywords
Ordinary <code>__input</code> , <code>__output</code> variables	Rename: now special globals

Earlier programs may need edits.

A Program with Text

```
__input = "Ada";  
name = input("Name? ");  
print("Hello, " + name + "!!");
```

Hello, Ada!

Literals and Values

Source	Value
"42"	String
42	Integer
""	Empty string
"a\nb"	Text containing a newline

The String Token

```
(r'"(?:\\[^\r\n] | [^"\\\r\n])*"', "string")
```

An escaped character or an ordinary character

```
token["value"] = decode_string(value, line, column)
```

Decoding the String

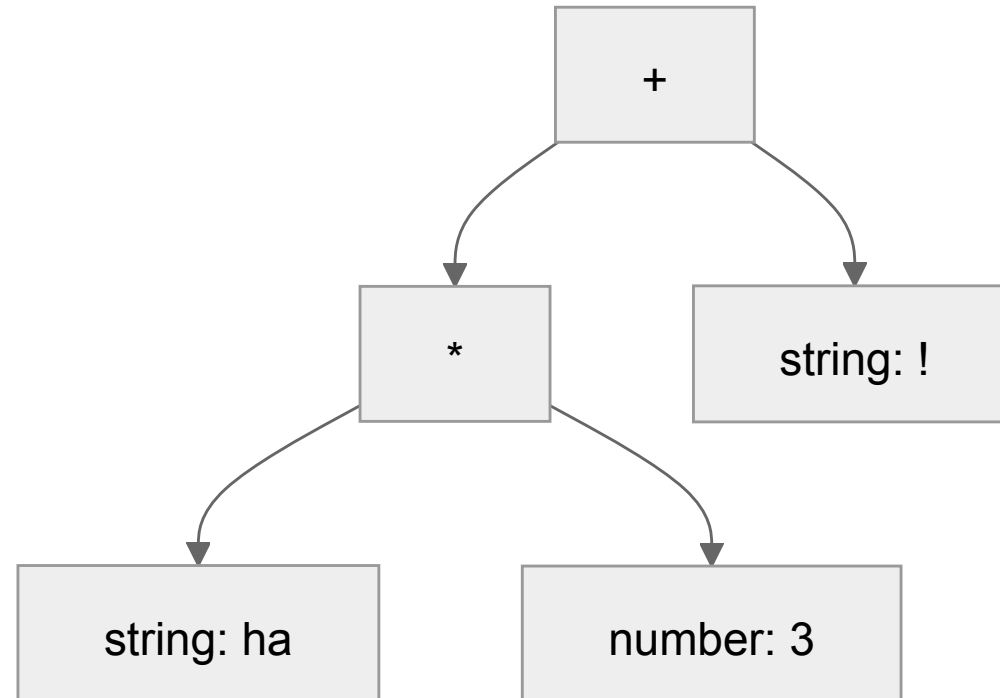
```
escapes = {"n": "\n", "t": "\t", "r": "\r", "'": "'", "\\": "\\"}\n
```

Ordinary character: copy it. Backslash: consume two characters.

Source characters	Value characters
\n	One newline
\"	One quotation mark
\\n	A backslash followed by n
\q	Syntax error

A String in an Expression Tree

"ha" * 3 + "!"



Operators and Types

Expression	Result
"ha" * 3	"hahaha"
3 * "ha"	"hahaha"
"3" + "4"	"34"
3 + 4	7
"ha" * 3.0	Type error

Input Returns Text

```
name = input();  
name = input("Your name? ");
```

Typing 42 produces "42".

Text to Number

Expression	Result
<code>number("42")</code>	Integer 42
<code>number("-3")</code>	Integer -3
<code>number(" .5 ")</code>	Float 0.5
<code>number("5.")</code>	Float 5.0

Number to Text

```
answer = 21 * 2;  
print(answer);  
print("Answer: " + string(answer));
```

```
42  
Answer: 42
```

Expression Forms

```
input_expression ::= "input" "(" [ expression ] ")"  
number_expression ::= "number" "(" expression ")"  
string_expression ::= "string" "(" expression ")"
```

```
answer = number(input()) * 2;
```

The Conversion Node

`number("21")`

```
{  
  "tag": "number_conversion",  
  "expression": {  
    "tag": "string", "value": "21"  
  }  
}
```

Print Is Still a Statement

```
print_statement ::= "print" "(" expression ")"
```

Source

Accepted?

`print(42)`

Yes

`print 42`

No

`value = print(42)`

No

Capturing Output

```
print(42);  
saved = __output;  
print("Captured: " + saved);
```

Variable after the program

Value

saved

"42"

__output

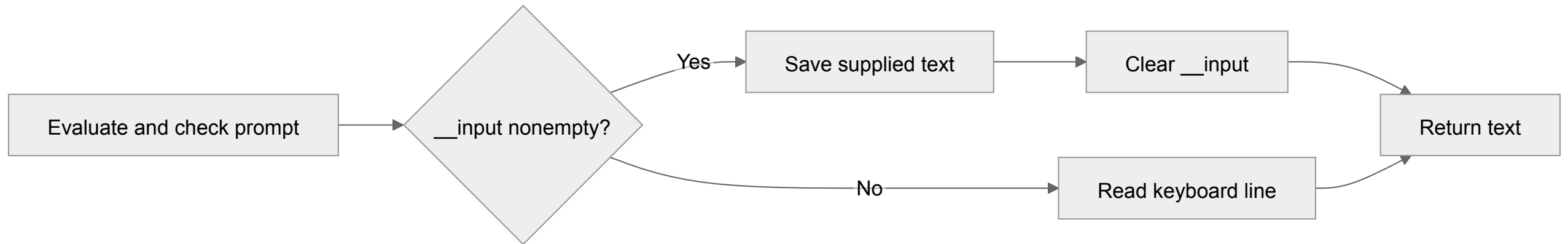
"Captured: 42"

Supplied Input Is Used Once

```
__input = "Ada";  
name = input();
```

After input	Value
name	"Ada"
__input	""

The Input Decision



A Repeatable Test

```
__input = "21";  
answer = number(input()) * 2;  
print(answer);
```

Expected: __input is "", __output is "42".

Errors at Different Stages

Example	Stage
Unterminated string	Tokenization
number()	Parsing
number("hello")	Evaluation
"ha" * 2.0	Evaluation

The Example and Tests

```
./vertex example.v < /dev/null  
bash run-tests.sh
```

Supplied input, clearing, captured output, conversions

Practice

Supply "18.5", add 2, and print "Total: 20.5".

Check the result and confirm that the input was consumed.